

Lesson 3 — Evaluating Regression Models

In Lesson 2, you trained your first model:

```
model = LinearRegression()

model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

Now we need to answer:

How good is our model?

A model can make predictions, but predictions alone don't tell us whether it's good or bad.

For regression, you'll learn four important metrics:

1. MAE — Mean Absolute Error
2. MSE — Mean Squared Error
3. RMSE — Root Mean Squared Error
4. R^2 — R-squared

1. First, understand the error

Suppose the actual price is:

```
Actual = 5
```

and your model predicts:

```
Predicted = 4
```

The error is:

```
Error = Actual - Predicted

Error = 5 - 4

Error = 1
```

Another example:

Actual	Predicted	Error
5	4	1
3	4	-1
8	6	2
2	3	-1

The problem is that some errors are positive and some negative.

If we simply add them:

```
1 + (-1) + 2 + (-1) = 1
```

That doesn't properly represent the total error.

So we need metrics that handle this.

2. MAE — Mean Absolute Error

MAE means:

Mean Absolute Error

“Absolute” means we ignore whether the error is positive or negative.

Suppose:

Actual	Predicted
5	4
3	4
8	6
2	3

Errors:

```
1
-1
2
-1
```

Absolute errors:

```
|1| = 1
|-1| = 1
|2| = 2
|-1| = 1
```

Then:

```
MAE = (1 + 1 + 2 + 1) / 4
MAE = 1.25
```

So the model's average error is 1.25 units.

3. Calculate MAE in Scikit-learn

Import:

```
from sklearn.metrics import mean_absolute_error
```

Then:

```
mae = mean_absolute_error(y_test, predictions)

print("MAE:", mae)
```

For example:

```
MAE: 0.53
```

The interpretation is:

On average, the predictions are off by about 0.53 target units.

4. MSE — Mean Squared Error

MSE means:

Mean Squared Error

Instead of taking the absolute value, we square the errors.

Example:

Error

1
-1
2
-1

Square them:

$1^2 = 1$
 $(-1)^2 = 1$
 $2^2 = 4$
 $(-1)^2 = 1$

Then:

$MSE = (1 + 1 + 4 + 1) / 4$

$MSE = 1.75$

In Scikit-learn:

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, predictions)

print("MSE:", mse)
```

5. Why does MSE punish large errors?

This is important.

Imagine:

Error = 2

Squared:

$2^2 = 4$

But:

```
Error = 10
```

Squared:

```
102 = 100
```

So a large error gets much more weight.

That's why MSE is particularly sensitive to large mistakes.

6. RMSE

RMSE means:

Root Mean Squared Error

It's simply the square root of MSE.

```
RMSE =  $\sqrt{\text{MSE}}$ 
```

For example:

```
MSE = 4
```

```
RMSE =  $\sqrt{4}$ 
```

```
RMSE = 2
```

In Scikit-learn:

```
import numpy as np
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, predictions)

rmse = np.sqrt(mse)

print("RMSE:", rmse)
```

Depending on your Scikit-learn version, you may also see newer convenience options for requesting RMSE directly, but `np.sqrt(mean_squared_error(...))` is perfectly clear and widely applicable.

7. Why use RMSE?

Because RMSE is in the same units as the target.

Suppose we're predicting house prices and:

```
MSE = 0.36
```

The squared unit can be difficult to interpret.

RMSE:

```
 $\sqrt{0.36} = 0.6$ 
```

Now we can say:

The typical prediction error is around 0.6 target units.

8. MAE vs MSE vs RMSE

Remember this:

Metric	Basic idea	Large errors
MAE	Average absolute error	Less sensitive
MSE	Average squared error	Very sensitive
RMSE	$\sqrt{\text{MSE}}$	Sensitive

For now:

MAE → easy to understand
MSE → strongly punishes big errors
RMSE → like MSE but back in target units

9. R^2 — R-Squared

Now the most important concept for today's lesson.

R^2 tells us roughly:

How much of the variation in the target is explained by the model.

Use:

```
from sklearn.metrics import r2_score

r2 = r2_score(y_test, predictions)

print("R²:", r2)
```

You might get:

R²: 0.60

Very roughly, that means the model explains about 60% of the variation in the target relative to the standard baseline used by R^2 .

10. Is $R^2 = 0.60$ good?

There's no universal answer.

It depends on:

- The dataset
- The problem
- The application
- The noise in the data
- What level of error is acceptable

For learning purposes, remember:

```
R2 closer to 1 → generally better
R2 around 0 → baseline-like performance
R2 < 0 → can be worse than the baseline
```

Don't think:

" $R^2 = 0.60$ means the model is 60% accurate."

That's incorrect.

R^2 is not accuracy.

11. Put Everything Together

Let's evaluate the Linear Regression model from Lesson 2.

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

mae = mean_absolute_error(y_test, predictions)
mse = mean_squared_error(y_test, predictions)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, predictions)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R2 :", r2)
```

You might get results approximately like:

```
MAE : 0.53
MSE : 0.52
RMSE: 0.72
R2 : 0.60
```

Your exact numbers may differ slightly depending on your environment and data handling.

12. How Do We Compare Two Models?

Suppose you train two models:

Model A

```
MAE = 0.55
RMSE = 0.75
R2 = 0.58
```

Model B

```
MAE = 0.48
RMSE = 0.68
R2 = 0.65
```

Model B is generally better here because:

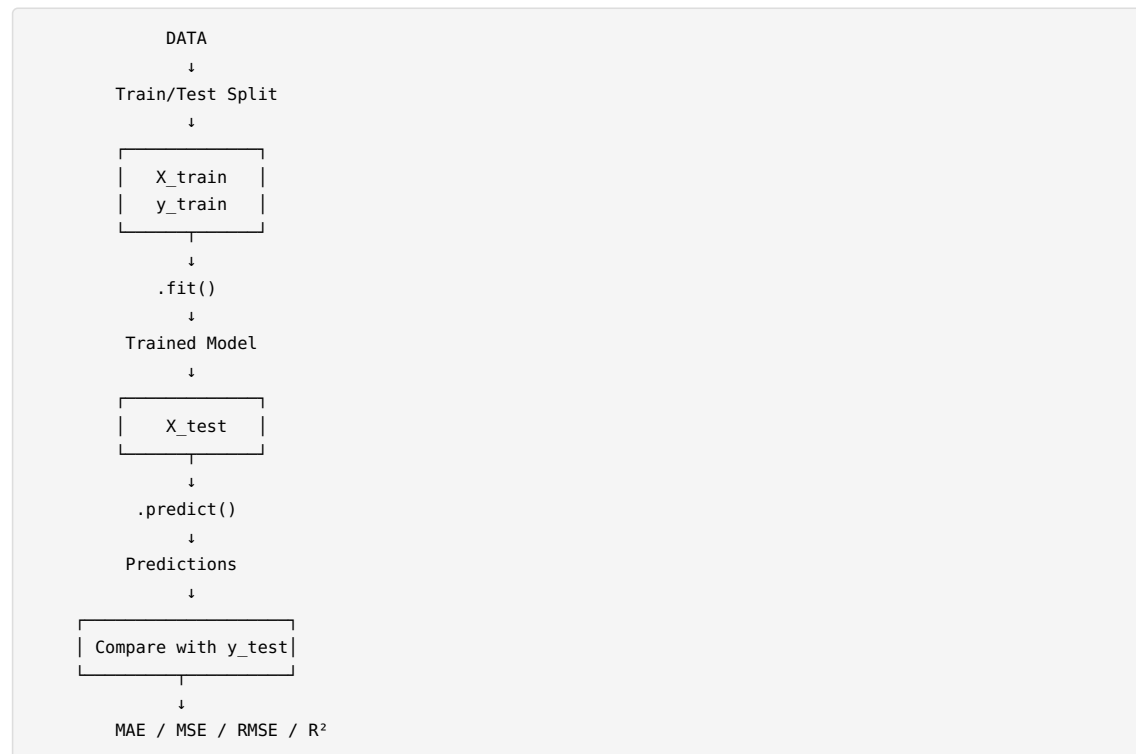
MAE ↓
RMSE ↓
R² ↑

So when comparing regression models:

Lower MAE/MSE/RMSE is generally better. Higher R² is generally better.

13. A Very Important Connection

Remember our ML workflow:



This is the basic train → predict → evaluate cycle.

📁 Lesson 3 Exercise

Run this in your notebook:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np
```

Load:

```
X, y = fetch_california_housing(return_X_y=True)
```

Split:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Train:

```
model = LinearRegression()

model.fit(X_train, y_train)
```

Predict:

```
predictions = model.predict(X_test)
```

Evaluate:

```
mae = mean_absolute_error(y_test, predictions)
mse = mean_squared_error(y_test, predictions)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, predictions)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R² :", r2)
```

Before Lesson 4, make sure you can answer:

1. Why can't we just look at the predictions?
2. What does MAE measure?
3. Why does MSE punish large errors?
4. Why do we take the square root of MSE to get RMSE?
5. Is $R^2 = 0.60$ the same as 60% accuracy?
6. For MAE, is 0.3 or 0.8 better?
7. For R^2 , is 0.8 or 0.4 generally better?